Week06

- wrap-up
  - array list runtime
  - double-ended queue (deque)
- midterm notes
- PostScript notes

WARMUP (until ~10:10am)
two silly questions:
- implement a stack using 1 queue
  - pseudocode is fine
  - (push and pop (and peek?))
  - what is the runtime (big O)?—slow is OK 😴
- implement a queue using 2 stacks
  - ...

wrap-up: array list

big-O runtime of

```
void add(ElementType e);
```

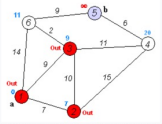
review: big-O runtime

- **big-O runtime** (**running time**, **time complexity**) gives us a big-picture idea of how long a function will take to run (execute, finish)
  - a function that operates on n *things*
    (**NOTE:** we assume n is LARGE)
    could be...
    - $\mathcal{O}(1)$ (**constant time**) 😀 FAST
    - $\mathcal{O}(\log_2 n)$ (**logarithmic time**, **log time**) 😄 fast
    - $\mathcal{O}(n)$ (**linear time**) 😐 meh
    - $\mathcal{O}(n^2)$ (**quadratic time**) 😟 slow
    - $\mathcal{O}(2^n)$ (**exponential time**) 😨 SLOW

big-O runtime

- **big-O runtime** can get complicated
  - sometimes there aren't just n *things*...
    - there are m thing A's, n thing B's, etc.
      - 🔮 **stay tuned for CSCI 256!**
  - sometimes we get more specific about *when* a function takes a certain length of time to run
    - in the **worst case**?
      - (if they don't say otherwise, this is what people mean)
    - in the **best case**?
    - ~~in the average case?~~
    - in the "long run"? (**amortized**)

Dijkstra's algorithm



Dijkstra's algorithm to find the shortest path between *a* and *b*. It picks the unvisited vertex with the lowest distance, calculates the distance through it to each unvisited neighbor, and updates the neighbor's distance if smaller. Mark visited (set to red) when done with neighbors.

Class	Search algorithm Greedy algorithm Dynamic programming[1]
Data structure	Graph Usually used with priority queue or heap for optimization[2][3]
Worst-case performance	$\Theta(\|E\| + \|V\| \log \|V\|)$[3]

best case and worst case runtime of adding to the back of an array list

best case

- the **best case** is the shortest time a function can possibly take to run
  - for adding an element to the back of an array list, this is when the array list's internal/private array still has at least one "empty slot"
    - in this case, we have to...
      - write an element to an array $\mathcal{O}(1)$
      - increment a counter + $\mathcal{O}(1)$
      - ---
      - $\mathcal{O}(1)$ 😃

worst case

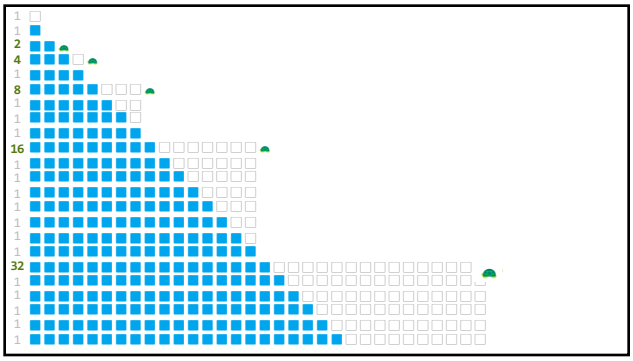
- the **worst case** is the longest time a function can possibly take to run
  - for adding an element to the back of an array list, this is when the array list's internal/private array is full (with n elements)
    - in this case, we have to...
      - allocate an array of $2n$ elements $\mathcal{O}(n)$
      - copy of over n elements + $\mathcal{O}(n)$
      - write an element to an array + $\mathcal{O}(1)$
      - increment a counter + $\mathcal{O}(1)$
      - updating reference to internal array + $\mathcal{O}(1)$
      - ---
      - $\mathcal{O}(n)$ 😃

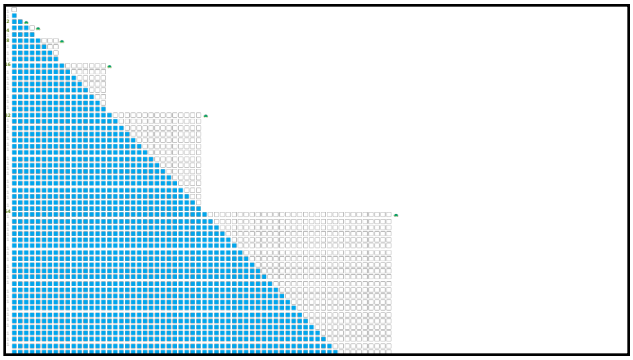
sometimes the best case and worst case are very different

amortized runtime of adding to the back of an array list

amortized

- the **amortized runtime** is how long a function takes (on average) "in the long run"
  - for adding an element to the back of an array list...
    - wasn't full 1
    - **was full** 2
    - **was full** 4
    - wasn't full 1
    - **was full** 8
    - wasn't full 1
    - wasn't full 1
    - wasn't full 1
    - **was full** 16
    - ...
    - 1

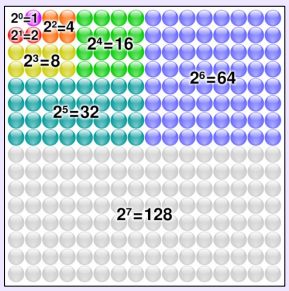




amortized runtime

- the **amortized runtime** is how long a function takes (on average) "in the long run"
  - for adding an element to the back of an array list...
    - $\frac{1+2+4+1+8+1+1+1+16+1+1+1+1+1+1+1+32+1+1+1+1+1+1+1+1}{n}$
    - $\frac{\mathcal{O}(1+\cdots+1)}{n} + \frac{\mathcal{O}(1+2+4+8+16+\cdots n)}{n}$
    - $\frac{\mathcal{O}(n)}{n} + \frac{\mathcal{O}(2n-1)}{n}$ *
    - $\frac{\mathcal{O}(n)}{n} + \frac{\mathcal{O}(n)}{n}$
    - $\mathcal{O}(1)$ 😳

*intuition



*proof

12 Answers

Sorted by: Highest score (default)

144

The binary expansion of $\sum_{k=0}^{n} 2^k$ is a string of $n+1$ 1's:

$$\underbrace{111\ldots111}_{n+1}$$

If I add a 1 to this number, what do I get?

$$\underbrace{1000\ldots000}_{n+1}$$

1 followed by $n+1$ 0's, hence 2^{n+1}. Therefore

$$\sum_{k=0}^{n} 2^k = 2^{n+1} - 1$$

the amortized run time is less *pessimistic* than worst case

the most relevant runtime depends on **context**

🧠 when might worst case be more relevant?

🧠 when might amortized be more relevant?

best case and worst case runtime of inserting into the *front* of an array list

best case and worst cast

- the **best case** of inserting an element into the front of an array list is when the array list's internal array still has at least one "empty slot"
  - in this case, we have to...
    - "move over" n elements $\mathcal{O}(n)$
    - write an element to an array + $\mathcal{O}(1)$
    - increment a counter + $\mathcal{O}(1)$
- the **worst case** of inserting an element into the front of an array list is when the array list's internal array is full
  - in this case, we have to...
    - allocate an array of $2n$ elements $\mathcal{O}(n)$
    - copy of over n elements + $\mathcal{O}(n)$
    - write an element to an array + $\mathcal{O}(1)$
    - increment a counter + $\mathcal{O}(1)$

lesson?: sometimes best case and worst case are the same

review: queue



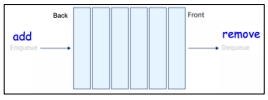
queue interface

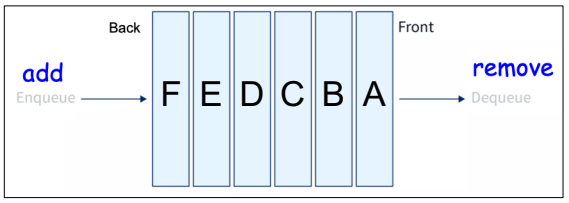
```
// Add (enqueue) a new element to the back of the queue.
void add(ElementType element);

// Remove (dequeue) the front
// element and return it.
ElementType remove();

// Peek (look) at the front element (without removing it)
// and return it.
ElementType peek();

// Returns the number of elements currently in the queue.
int size();
```






```
queue.add(A)
queue.add(B)
queue.add(C)
...
```

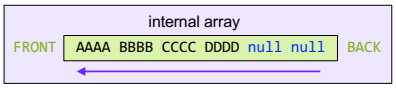
wrap-up: deque

(double-ended queue)

naive deque implemenation

implementing a deque using a left-aligned, left-facing internal array

- `addFront(...)`
- `removeFront()`
- `addBack(...)`
- `removeBack()`



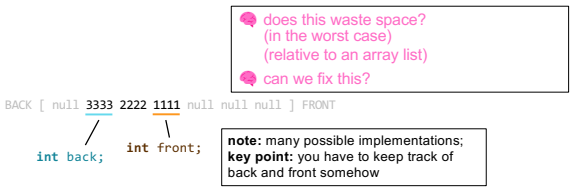
- 🧠 what is the **amortized runtime** of these functions if we were to implement the deque interface using an array list?
  - 😀 `O(n) addFront(...) and removeFront()`
  - 😀 `O(1) addBack(...) and removeBack()`
- 🧠 can we do better?

wonderful deque implemenation

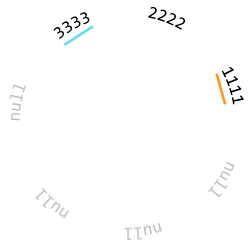
neat idea: store the queue in the *middle*

```
BACK [ null null null null null null null ] FRONT
                                                    // addBack(1111)
BACK [ null null null 1111 null null null ] FRONT
                                                    // addBack(2222)
BACK [ null null 2222 1111 null null null ] FRONT
                                                    // addBack(3333)
BACK [ null 3333 2222 1111 null null null ] FRONT
                                                    // removeFront()
BACK [ null 3333 2222 null null null null ] FRONT
                                                    // addFront(4444)
BACK [ null 3333 2222 4444 null null null ] FRONT
                                                    // addFront(5555)
BACK [ null 3333 2222 4444 5555 null null ] FRONT
                                                    // removeBack()
BACK [ null null 2222 4444 5555 null null ] FRONT
```

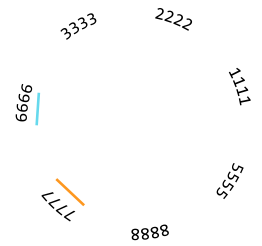
neat idea: store the queue in the *middle*



extension: treat the array as circular

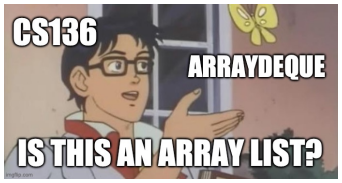


extension: treat the array as circular



big picture lesson

once you get past CS136,
data structures are actually kind of
rich and complicated and messy



midterm notes

about english

- if you don't understand what a question is asking, **please ask me during the test!**

things to know

- if, if, if vs. if, else if, else if
- Cow canvas coordinates
  - drawCircle
  - drawLine
  - drawing pictures with a pencil
- accessing arrays

midterm questions?

where did the PostScript Starter Code come from?

the big decision in the Starter Code is that we're going to chop the program into a list of "tokens"

and each token could be a boolean, double, String, or list

in Python, a variable's type is **dynamic**

```
token = True
print(type(token)) # <class 'bool'>

token = 5.0
print(type(token)) # <class 'float'>

token = "exch"
print(type(token)) # <class 'str'>

token = ["exch", 0.0, "add"]
print(type(token)) # <class 'list'>
```

in Java, a variable's type is **static**

0	false	0.0	null	null

```
class Token {
    int type;
    private boolean _valueIfTypeBoolean;
    private double _valueIfTypeDouble;
    private String _valueIfTypeString;
    private ArrayList<Token> _valueIfTypeList;
    static final int TYPE_BOOLEAN = 0;
    static final int TYPE_DOUBLE = 1;
    static final int TYPE_STRING = 2;
    static final int TYPE_LIST = 3;
}
```

examples

- `new Token(true)`

0	true	0.0	null	null

- `new Token(5.0)`

1	false	5.0	null	null

- `new Token("exch")`

2	false	0.0	0x02343af34...	null

"exch"

in Java, a variable's type is **static**

```
// Create a String-type Token.
Token token = new Token();
token.type = Token.TYPE_STRING;
token._valueIfTypeString = "exch";
```

```
// Get the value of a String-type token.
ASSERT(token.type == Token.TYPE_STRING); // check type!
String string = token._valueIfTypeString;
```

this usage code makes me sad ☹️
let's write some ✨ **functions** ✨!

in Java, a variable's type is **static**

```
// Create a String-type Token.
Token token = new Token("exch");
```

```
// Get the value of a String-type token.

String string = token.getString();
```

all better 😀

tl;dr

use these functions 🙄 👍

Token

- Simple class that can represent any type of token.
  - Convenient getters with assert statements.
  - Static method to "tokenize" a PostScript program (turn it into a list of tokens).

```
class Token {
    int type; // type of Token; can be Token.TYPE_BOOLEAN, Token.TYPE_DOUBLE, Token.TYPE_STRING,

    Token(boolean          value); // creates a new boolean type token with given value
    Token(double           value); // creates a new double  type token with given value
    Token(String           value); // creates a new string  type token with given value
    Token(ArrayList<Token> value); // creates a new list    type token with given value

    boolean          getBoolean(); // gets token's value as a  boolean          (crashes if not
    double           getDouble();  // gets token's value as a  double           (crashes if not
    String           getString();  // gets token's value as a  String           (crashes if not
    ArrayList<Token> getList();    // gets token's value as an ArrayList<Token> (crashes if not

    static ArrayList<Token> tokenize(String postScriptProgram); // turns PostScript program into
}
```

and these functions!! 🙄 👍

Interpreter

- Class that wraps around the stack and the map.
  - Method to interpret a PostScript program that helpfully prints out the stack and map for you.
  - Method to interpret a single token (the only thing you need to implement).
  - Convenient methods for working with the stack and map with assert statements.

```
class Interpreter {
    void interpret(ArrayList<Token> tokens); // interprets a list of tokens
    void interpret(Token token); // interprets a single token

    Token            stackPopToken();   // pops a Token off of the stack
    boolean          stackPopBoolean(); // pops a Token off of the stack and gets its value as a
    double           stackPopDouble();  // pops a Token off of the stack and gets its value as a
    String           stackPopString();  // pops a Token off of the stack and gets its value as a
    ArrayList<Token> stackPopList();    // pops a Token off of the stack and gets its value as a
    void stackPush(Token            token); // pushes a token onto the stack
    void stackPush(boolean          value); // creates a new boolean type token with given value
    void stackPush(double           value); // creates a new double  type token with given value
    void stackPush(String           value); // creates a new string  type token with given value
    void stackPush(ArrayList<Token> value); // creates a new list    type token with given value

    void    mapPut(String key, Token value); // puts a key-value pair into the map
    Token   mapGet(String key); // for the given key, looks up the corresponding value in the ma
    boolean mapContainsKey(String key); // returns whether the map contains a key-value pair wit
}
```

```
// Pop a double off the stack
ASSERT(_stack.size() > 0);
Token token = _stack.pop();
double num = token.getDouble();
```

```
// Push a double onto the stack.
_stack.push(new Token(5.0));
```

🙁

```
// Pop a double off the stack


double num = stackPopDouble();
```

```
// Push a double onto the stack.
stackPush(5.0);
```

🙄